# V-Sem Mini Project Report on

# FARM2HOME – DIRECT FARMER TO CONSUMER WEB APPLICATION

**Submitted in partial fulfilment of the requirement for the award of the degree of**

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE & ENGINEERING (AI/ML)**


**Submitted by:**

| **Student Name:** | **Roll No:** |
|---|---|
| CHAMARTHI MANIKANTA | 25985A4205 |
| DADI SAI DINENDRA | 25985A4207 |
| KUDUMULA SAI KRISHNA | 25985A4219 |
| SUNAPATHI JASWANTH | 25985A4236 |
| MEKA SRI KOWSHIK | 24981A422N |

***Under the Guidance of***
**K SANDEEP KUMAR**
**Assistant Professor**






**Raghu Engineering College**
**Department of Computer Science and Engineering (AI/ML)**
**Dakamarri, Visakhapatnam**

**2026-27**

# CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the project report entitled **"Farm2Home – Direct Farmer to Consumer Web Application"** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering **(AI/ML)** in the Department of Computer Science and Engineering**(AI/ML)** of the **Raghu Engineering College** shall be carried out by the undersigned under the supervision of **K SANDEEP KUMAR, Assistant professor**, Department of Computer Science and Engineering**(AI/ML)**, **Raghu Engineering College** Visakhapatnam.

| **Name:** | **Roll No:** | **Signature** |
|---|---|---|
| CHAMARTHI MANIKANTA | 25985A4205 | |
| DADI SAI DINENDRA | 25985A4207 | |
| KUDUMULA SAI KRISHNA | 25985A4219 | |
| SUNAPATHI JASWANTH | 25985A4236 | |
| MEKA SRI KOWSHIK | 24981A422N | |

The above-mentioned students shall be working under the supervision of the undersigned on the **"Farm2Home – Direct Farmer to Consumer Web Application"**

| **Project Guide** | **Head of the Department CSE(AI/ML)** |
|---|---|
| **K SANDEEP KUMAR** | **Dr. G Sridevi** |
| **Assistant Professor** | **Associate Professor** |

## Examination

| **Name of the Examiners:** | **Signature with Date** |
|---|---|
| 1. **Dr. G Sridevi** | |
| 2. | |

## ACKNOWLEDGEMENT

We express my sincere gratitude to the management, Principal, Head of the Department, project guide, faculty members and staff of Raghu Engineering College for their valuable guidance and support during the development of this mini project, especially our project guide K Sandeep Kumar and our Head of the Department Dr. G Sridevi. We also thank my classmates and friends for their suggestions and encouragement. Finally, we thank my family for their continuous support throughout the project.



**Team members**

CHAMARTHI MANIKANTA (25985A4205)

DADI SAI DINENDRA (25985A4207)

KUDUMULA SAI KRISHNA (25985A4219)

SUNAPATHI JASWANTH (25985A4236)

MEKA SRI KOWSHIK (24981A422N)

# ABSTRACT

Farm2Home is a web-based platform developed using HTML, CSS, and JavaScript that connects farmers directly with consumers for a simple, interactive, and convenient online shopping experience. The system allows users to browse, search, and view fresh farm products with details such as price, category, availability, and product information. Consumers can create accounts, manage their profiles, add products to their Wishlist and Shopping Cart, and place orders through the Checkout module. The platform also provides a farmer-oriented interface to display and manage agricultural products, helping reduce the gap between farmers and customers. Using browser-based local storage and session storage, the application maintains user, product, cart, wishlist, and order information without requiring a server-side backend. The project demonstrates DOM manipulation, event handling, responsive design, data persistence, and dynamic web interfaces to build a functional Farm2Home application for direct farm-to-consumer commerce.

# TABLE OF CONTENTS

**1. INTRODUCTION ..... 7**
1.1 Purpose ..... 7
1.2 System Overview ..... 7
1.3 Problem Statement ..... 8
1.4 DFD Modelling of Problem ..... 8
1.5 Goal & Vision ..... 9
**2. REQUIREMENTS SPECIFICATIONS ..... 11**
2.1 User Characteristics ..... 11
2.2 Functional Requirements ..... 11
2.3 Dependencies ..... 14
2.4 Performance Requirements ..... 14
2.5 Hardware Requirements ..... 15
2.6 Constraints & Assumptions ..... 15
**3. DESIGN ..... 17**
3.1 Function Oriented Design for procedural approach ..... 17
3.2 Database Design ..... 18
3.3 GUI Design for front-end ..... 20
**4. CODING ..... 21**
4.1 Storage Helper (storage.js) ..... 21
4.2 Registration and Login (auth.js) ..... 21
4.3 Loading, Rendering and Searching Products (products.js) ..... 22
4.4 Wishlist (wishlist.js) ..... 23
4.5 Shopping Cart (cart.js) ..... 23
4.6 Checkout and Order Placement (checkout.js) ..... 24
4.7 Farmer Product Management (farmer.js) ..... 25
4.8 Responsive Styling (style.css) ..... 25
**5. TESTING ..... 26**
5.1 Test Plan ..... 26
5.2 Test Report ..... 26
**6. INSTALLATION INSTRUCTIONS ..... 29**
**7. END-USER INSTRUCTIONS ..... 30**
7.1 For the Consumer ..... 30
7.2 For the Farmer ..... 30
**8. FUTURE WORK ..... 32**
**9. SUMMARY ..... 33**
**10. REFERENCES ..... 34**

# LIST OF FIGURES

Figure 1. Level-0 DFD (Context Diagram) of Farm2Home

Figure 2. Level-1 DFD of Farm2Home

Figure 3. DFD Process 2 – Cart, Wishlist and Checkout

Figure 4. DFD Process 3 – Farmer Product Management

Figure 5. Module Structure of the Application

Figure 6. Screenshot of Home / Product Listing Page

Figure 7. Screenshot of Login and Registration Page

Figure 8. Screenshot of Cart, Wishlist and Checkout Pages

Figure 9. Screenshot of Farmer Dashboard

Figure 10. Test Report

# 1. INTRODUCTION

This section gives a scope description and overview of everything included in this Project Report. The purpose of the document is described, and the system overview along with the goal and vision of the Farm2Home application are listed.

## 1.1 Purpose

The purpose of this document is to give a detailed description of the Farm2Home project. It illustrates the purpose and complete declaration for the development of the system. It also explains the system constraints, the interfaces between modules and the way the browser storage is used to persist data. This document is primarily intended for anyone who wants an overview of how Farm2Home works, its outcomes and its possible usages in the future, including faculty evaluators, fellow students who want to extend the project, and farmers or consumers who would like to understand the idea behind the platform.

The report follows the standard mini project format prescribed by the Department of Computer Science and Engineering (AI/ML): Introduction, Requirements Specification, Design, Coding, Testing, Installation Instructions, End-User Instructions, Future Work, Summary and References.

## 1.2 System Overview

Farm2Home is a front-end web application built with the three core web technologies: **HTML** for the structure of pages, **CSS** for styling and responsive layout, and **JavaScript** for behaviour. The application lets a consumer browse a catalogue of fresh farm products such as vegetables, fruits, grains, dairy and spices. Every product card shows the product name, image, price, category, availability and a short description.

A consumer can register an account, log in, manage a profile, add products to a **Wishlist** for later, add products to a **Shopping Cart**, change quantities, and finally place an order through the **Checkout** module. A second type of user, the **farmer**, has an interface in which agricultural products can be displayed, added, edited and removed, so that the produce reaches the customer without a middleman.

The application does not need a server-side backend or a database server. All information about users, products, the cart, the wishlist and orders is kept in the browser using the Web Storage API:

- **localStorage** keeps data that must survive after the browser is closed (registered users, products, wishlist, orders).
- **sessionStorage** keeps data that is valid only for the current session (currently logged-in user, temporary checkout details).

Because of this, the whole project can be opened simply by double-clicking index.html, which makes it easy to demonstrate and to deploy on any static hosting service.

## 1.3 Problem Statement

Indian farmers usually sell their produce through a chain of agents, commission men and retailers. Each stage adds its own margin. As a result the farmer receives only a fraction of the price paid by the consumer, while the consumer pays a high price for produce that may already be several days old. Small and marginal farmers have no direct digital channel to show their products to nearby customers, and many consumers do not have a simple way to know which fresh products are available, at what price, and from whom.

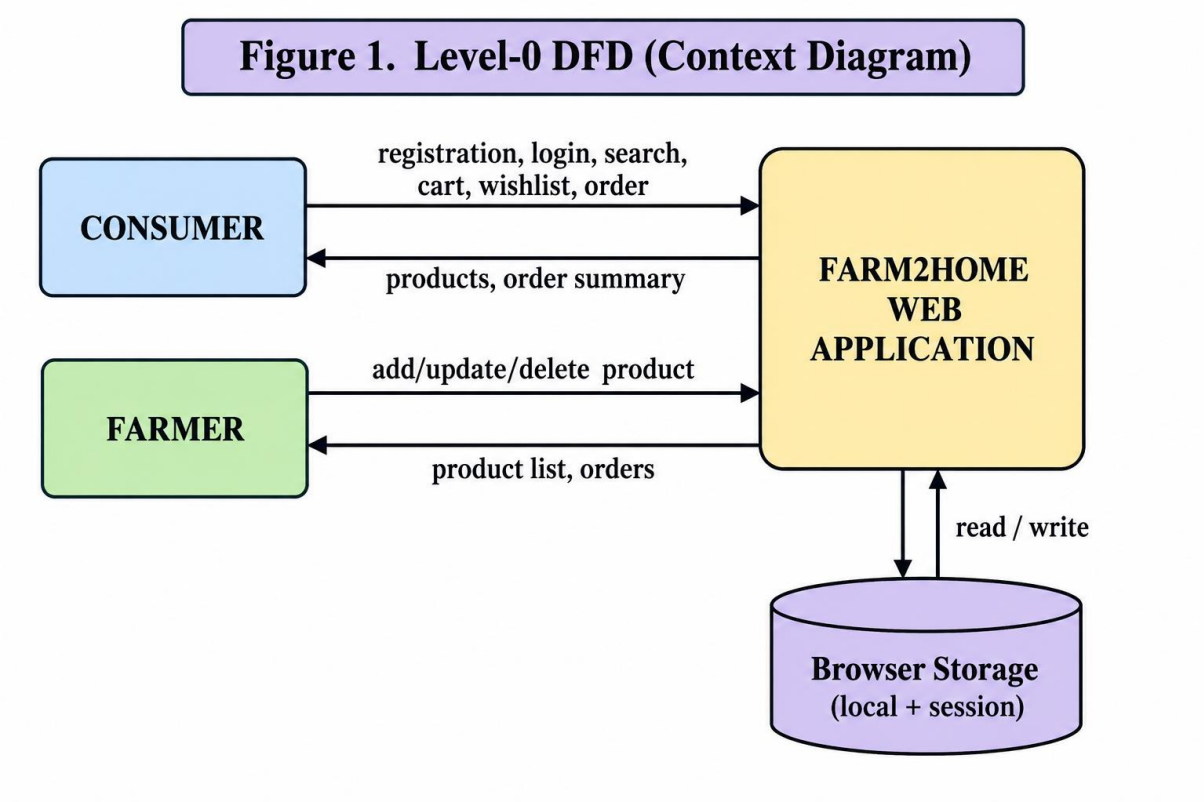
The main problems identified are:

1. Lack of a direct link between farmers and consumers.
2. Low share of the final price reaching the farmer.
3. Lack of price and availability transparency for the consumer.
4. Difficulty in searching and comparing fresh products at one place.
5. Complex e-commerce platforms that need a heavy server set-up and are difficult for students to understand and extend.

Farm2Home solves this problem by providing a simple, interactive and responsive web interface in which products from farmers are listed, searched and ordered directly, using only client-side technologies.

## 1.4 DFD Modelling of Problem

A Data Flow Diagram (DFD) shows how data moves through a system. At the highest level (Level-0 or context diagram) Farm2Home is a single process that interacts with two external entities: the **Consumer** and the **Farmer**. The browser storage acts as the data store.




At Level-1 the system is decomposed into the following processes:

| Process | Name | Input | Output |
|---|---|---|---|
| 1.0 | User Management | Registration / login details | Session user, profile |
| 2.0 | Product Browsing & Search | Search keyword, category filter | Filtered product list |
| 3.0 | Wishlist & Cart Management | Product id, quantity | Updated wishlist and cart |
| 4.0 | Checkout & Order Management | Delivery details, cart | Order record, confirmation |
| 5.0 | Farmer Product Management | Product details | Updated product catalogue |

```
Figure 2. Level-1 DFD

Consumer --(credentials)--> [1.0 User Management] <--> D1: Users
Consumer --(keyword)------> [2.0 Browse & Search]  <--> D2: Products
Consumer --(product id)---> [3.0 Cart & Wishlist]  <--> D3: Cart / Wishlist
Consumer --(address)------> [4.0 Checkout]         <--> D4: Orders
Farmer   --(product data)-> [5.0 Product Mgmt]     <--> D2: Products
```

The data stores are D1 Users, D2 Products, D3 Cart/Wishlist and D4 Orders. All of them are represented as keys in the browser storage, as explained in Chapter 3.

## 1.5 Goal & Vision

The goal of Farm2Home is to provide a simple, interactive and convenient online shopping experience through which farmers and consumers are connected directly. The system aims to:

- Reduce the gap between the farmer and the customer by removing intermediaries.
- Give the consumer clear information such as price, category and availability of every product.
- Provide a responsive interface that works on desktop, tablet and mobile screens.
- Demonstrate the practical use of DOM manipulation, event handling, data persistence and dynamic interface generation in JavaScript.

The vision is that this front-end prototype can become the basis for a full-scale farm-to-consumer marketplace. By replacing the browser storage with a server-side API and a database, and by adding online payments and delivery tracking, the same interface can serve real farmers and customers. It also serves as a learning model for students who wish to understand how a complete e-commerce workflow (browse, wishlist, cart, checkout) is built from basic web technologies.

# 2. REQUIREMENTS SPECIFICATIONS

This chapter lists the users of the system, the functions the system must perform, the technologies it depends on, and the limits within which it works.

## 2.1 User Characteristics

There are three types of users that interact with the system:

- **Consumer (Customer)** - browses products, maintains an account and profile, uses the wishlist and cart and places orders.
- **Farmer** - uses the farmer-oriented interface to display and manage agricultural products.
- **Guest (unregistered visitor)** - can browse and search products but must register or log in to add products to the cart, the wishlist or to place an order.

All the users access the application through a web browser on a PC, laptop, tablet or mobile phone. No technical knowledge is required. The interface uses familiar e-commerce elements - product cards, search box, category filters, cart icon with item count and a heart icon for the wishlist - so that a first-time user can understand the system without training.

The consumer opens the portal, sees the list of products and may search using the search box. When the user clicks a product the details are shown. After login the user may add the product to the cart or wishlist. In the checkout page the user enters the delivery details and confirms the order, after which an order summary is displayed.

## 2.2 Functional Requirements

**User Class 1 - The Consumer**

**Functional Requirement 1.1 - Registration**

- Actor: Consumer
- Input: Name, e-mail, phone number, password
- Description: A new user can create an account. The application validates the fields (non-empty fields, valid e-mail format, minimum password length) and checks that the e-mail is not already registered. On success, the user record is stored in localStorage.

**Functional Requirement 1.2 - Login and Logout**

- Actor: Consumer / Farmer
- Input: E-mail and password
- Description: The entered credentials are compared with the stored user records. If they match, the user object is saved in sessionStorage as the current session user and the navigation bar changes to show the user name and Logout option. Logout clears the session.

**Functional Requirement 1.3 - Profile Management**

- Actor: Consumer
- Input: Name, phone, address
- Description: A logged-in user can view and update the profile information. The updated record replaces the earlier record in localStorage.

**Functional Requirement 1.4 - Browse, Search and Filter Products**

- Actor: Consumer / Guest
- Input: Search keyword, category, price range
- Description: The product list is displayed as cards. The user can search by name and filter by category. The list is regenerated dynamically without reloading the page.

**Functional Requirement 1.5 - View Product Details**

- Actor: Consumer / Guest
- Input: Selection of a product card
- Description: The name, image, price, unit, category, availability, farmer name and description of the selected product are displayed.

**Functional Requirement 1.6 - Wishlist**

- Actor: Consumer
- Input: Add / remove action on a product
- Description: Products can be saved to a wishlist and later moved to the cart. The wishlist is saved per user.

**Functional Requirement 1.7 - Shopping Cart**

- Actor: Consumer
- Input: Add, increase, decrease, remove

- Description: Items can be added to the cart, quantities can be changed and items can be removed. The subtotal, delivery charge and grand total are recalculated after every change.

**Functional Requirement 1.8 - Checkout and Order Placement**

- Actor: Consumer
- Input: Delivery name, address, phone, payment mode (Cash on Delivery)
- Description: The checkout module validates delivery details, creates an order record containing the items, total, date and status, stores it in localStorage, empties the cart and shows an order confirmation with an order id.

**Functional Requirement 1.9 - Order History**

- Actor: Consumer
- Input: Selection of the My Orders page
- Description: The user can see all earlier orders with date, items, amount and status.

**User Class 2 - The Farmer**

**Functional Requirement 2.1 - Display Products**

- Actor: Farmer
- Description: The farmer dashboard lists all the products added by that farmer along with price and availability.

**Functional Requirement 2.2 - Add, Edit and Delete Products**

- Actor: Farmer
- Input: Product name, category, price, unit, quantity available, description, image
- Description: The farmer can add a new product, edit its details or delete it. The change immediately reflects on the consumer product page because both share the same product store.

**Functional Requirement 2.3 - Update Availability**

- Actor: Farmer
- Input: Quantity / in-stock flag
- Description: The farmer can mark a product as In Stock or Out of Stock so the consumer sees correct availability.

## 2.3 Dependencies

Farm2Home is a purely client-side application. It does not need a web server framework or database server. It depends on the following technologies and browser features:

- **HTML5** - semantic structure of all pages.
- **CSS3** - Flexbox, Grid and media queries for responsive design.
- **JavaScript (ES6)** - application logic, DOM manipulation and event handling.
- **Web Storage API** - localStorage and sessionStorage for persistence.
- **JSON** - JSON.stringify() and JSON.parse() are used to store objects and arrays as strings.
- **Google Fonts / Font Icons (optional)** - typography and icons.
- **Visual Studio Code with Live Server (optional)** - development environment.

There are no third-party JavaScript libraries or frameworks required, which keeps the page load fast and the code easy to understand.

## 2.4 Performance Requirements

Since all the data is read from and written to the browser storage, no network latency is involved after the page is loaded. The performance requirements are:

- A page should load in less than three seconds on a normal broadband or mobile connection.
- Search and filter results should appear instantly (under 200 ms) for a catalogue of a few hundred products.
- Cart totals must be recalculated immediately after each change.
- The layout should adapt without horizontal scrolling on screens from 320 px to 1920 px wide.
- Data stored in localStorage should remain available after the browser is closed and reopened.

The browsers limit localStorage to roughly 5 MB per origin, which is enough for the text data of the prototype; product images are therefore kept as small files or links instead of encoded image data.

## 2.5 Hardware Requirements

To use Farm2Home a user needs only a PC, laptop, tablet or mobile phone with an updated web browser.

- Desktop browsers: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera.
- Mobile browsers: Chrome for Android, Safari on iOS, Samsung Internet.

For development and demonstration the following minimum configuration is recommended:

| Component | Minimum Requirement |
|---|---|
| Processor | Intel Core i3 or equivalent |
| RAM | 4 GB |
| Free disk space | 500 MB |
| Operating System | Windows 10 / Ubuntu / macOS |
| Software | Modern web browser, VS Code or any text editor |
| Display | 1366 x 768 or higher |

No server hardware is needed because the application is a static site. It can be hosted on any static hosting service such as GitHub Pages or Netlify.

## 2.6 Constraints & Assumptions

**Constraints**

- Since there is no server-side backend, data is stored only in the browser of the user. Data entered on one device or browser is not visible on another.
- localStorage is limited in size (about 5 MB) and is not encrypted. Therefore passwords must not be treated as secure in the prototype and real payment information is not collected.
- Payment is limited to a simulated Cash on Delivery option; no payment gateway is integrated.
- Clearing the browser data will remove all stored records.

**Assumptions**

- The user has a modern browser with JavaScript and Web Storage enabled.
- The input given by the user is meaningful (valid e-mail, numeric phone number and so on).
- The product catalogue is initialised with sample data on first load and farmers add further products through their interface.

- The application currently supports the English language and prices in Indian Rupees.
- Farm2Home is an academic prototype and not a production commerce system.
- LocalStorage is browser-specific and does not synchronise data across devices or users.
- Checkout and order tracking are simulated; no actual payment or delivery booking is performed
- Data displayed by the prototype depends on the sample product and farmer information included in the frontend.
- Secure accounts, role-based access, server-side validation and central data management are outside the current implementation

# 3. DESIGN

This chapter describes how Farm2Home is designed: the procedural (function-oriented) structure of the code, the way data is organised in the browser storage, and the graphical user interface.

## 3.1 Function Oriented Design for procedural approach

The application is written in a procedural style: the program is divided into small functions, each performing one clear task, and the functions are grouped into modules (JavaScript files). Event handlers call these functions in response to user actions such as clicking a button or typing in the search box.

```
Figure 5. Module Structure of the Application

Farm2Home/
|-- index.html          (home + product listing)
|-- login.html          (login / registration)
|-- profile.html        (profile management)
|-- wishlist.html       (wishlist page)
|-- cart.html           (shopping cart)
|-- checkout.html       (checkout + order confirmation)
|-- orders.html         (order history)
|-- farmer.html         (farmer dashboard)
|-- css/
|     |-- style.css     (common styles, responsive rules)
|-- js/
|     |-- storage.js    (read / write helpers)
|     |-- auth.js       (register, login, logout, session)
|     |-- products.js   (load, render, search, filter)
|     |-- wishlist.js   (add / remove / list wishlist)
|     |-- cart.js       (add / update / remove / totals)
|     |-- checkout.js   (validation, order creation)
|     |-- farmer.js     (add / edit / delete products)
|-- images/             (product images)
```

**Level-1 DFD processes and the functions that implement them**

| Process | Main functions |
|---|---|
| 1.0 User Management | registerUser(), loginUser(), logoutUser(), getSession(), updateProfile() |
| 2.0 Browse & Search | loadProducts(), renderProducts(), applyFilters() |
| 3.0 Cart & Wishlist | addToCart(), updateQuantity(), calculateTotal(), toggleWishlist() |
| 4.0 Checkout | validateCheckout(), placeOrder(), generateOrderId() |
| 5.0 Farmer Management | addProduct(), editProduct(), deleteProduct(), setAvailability() |

```
Figure 3. DFD Process 2 - Cart, Wishlist and Checkout

Consumer --product id--> [3.1 Add to Cart] --> D3: Cart
Consumer --product id--> [3.2 Toggle Wishlist] --> D3: Wishlist
D3: Cart --items--> [3.3 Calculate Total] --total--> Consumer
Consumer --delivery details--> [4.1 Validate] --valid--> [4.2 Place Order]
[4.2 Place Order] --order record--> D4: Orders
[4.2 Place Order] --clear--> D3: Cart
[4.2 Place Order] --confirmation--> Consumer
```

```
Figure 4. DFD Process 3 - Farmer Product Management

Farmer --product details--> [5.1 Validate] --> [5.2 Save Product] --> D2: Products
Farmer --edit / delete request--> [5.3 Modify Product] <--> D2: Products
D2: Products --list--> [5.4 Display Products] --> Farmer
```

**Flow of a typical purchase**

1. The page loads and loadProducts() reads the product list from localStorage (it creates the default list on first use).
2. renderProducts() builds one card for each product using DOM methods and inserts the cards into the product grid.
3. The consumer logs in; loginUser() stores the session user in sessionStorage.
4. The consumer clicks Add to Cart; addToCart() updates the cart saved for that user and updates the cart count badge in the header.
5. In the cart page calculateTotal() computes the subtotal and total.
6. In checkout, validateCheckout() verifies the form and placeOrder() saves the order, clears the cart and shows the confirmation.

## 3.2 Database Design

Farm2Home does not use a database server. The "database" is the Web Storage of the browser, which is a key-value store. Browser storage can save only strings, so every JavaScript object or array is converted using JSON.stringify() before saving and converted back using JSON.parse() after reading. Every operation that changes data is written back immediately so the data is not lost.

localStorage persists after the browser is closed. sessionStorage is cleared when the tab or browser window is closed. The keys used by the application are:

| Storage | Key | Type | Description |
|---|---|---|---|
| localStorage | f2h_users | Array of user objects | All registered users |
| localStorage | f2h_products | Array of product objects | Product catalogue |
| localStorage | f2h_cart_userId | Array of cart items | Cart of one user |

| Storage | Key | Type | Description |
|---|---|---|---|
| localStorage | f2h_wishlist_userId | Array of product ids | Wishlist of one user |
| localStorage | f2h_orders | Array of order objects | All placed orders |
| sessionStorage | f2h_session | User object | Currently logged-in user |
| sessionStorage | f2h_checkout | Object | Temporary checkout details |

### User object

| Field | Description |
|---|---|
| id | Unique id of the user |
| name | Full name |
| email | E-mail address (used as login id, unique) |
| phone | Mobile number |
| password | Password entered during registration (prototype only) |
| role | consumer or farmer |
| address | Delivery address |

### Product object

| Field | Description |
|---|---|
| id | Unique id of the product |
| name | Product name, e.g. Tomato |
| category | Vegetables / Fruits / Grains / Dairy / Spices |
| price | Price per unit in Rupees |
| unit | kg / litre / dozen / piece |
| stock | Quantity available |
| available | true or false |
| farmer | Name of the farmer |
| image | Path of the image |
| description | Short description |

### Order object

| Field | Description |
|---|---|
| orderId | Unique order id such as F2H-1023 |
| userId | Id of the consumer |
| items | Array of {productId, name, price, qty} |
| total | Grand total in Rupees |
| address | Delivery address |
| payment | Payment mode (Cash on Delivery) |
| date | Date and time of the order |
| status | Placed / Packed / Delivered |

```
Entity relationships

USER (1) ---< places >--- (M) ORDER (1) ---< contains >--- (M) ORDER ITEM
USER (1) ---< owns   >--- (1) CART / WISHLIST
FARMER (1) -< lists  >--- (M) PRODUCT (1) ---< appears in >--- (M) ORDER ITEM
```

## 3.3 GUI Design for front-end

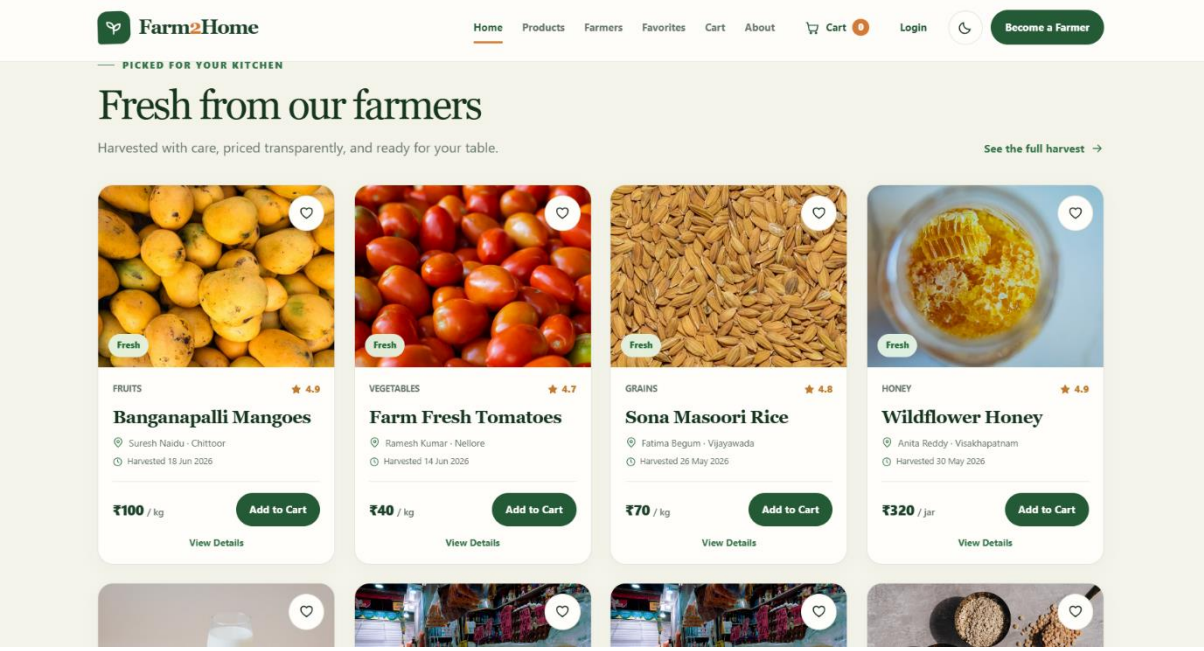
The interface is designed to be simple, clean and responsive. A green and white colour theme is used to reflect agriculture and freshness. All pages share the same header (logo, search box, navigation links, wishlist icon and cart icon with item count) and the same footer, so the user always knows where to navigate.

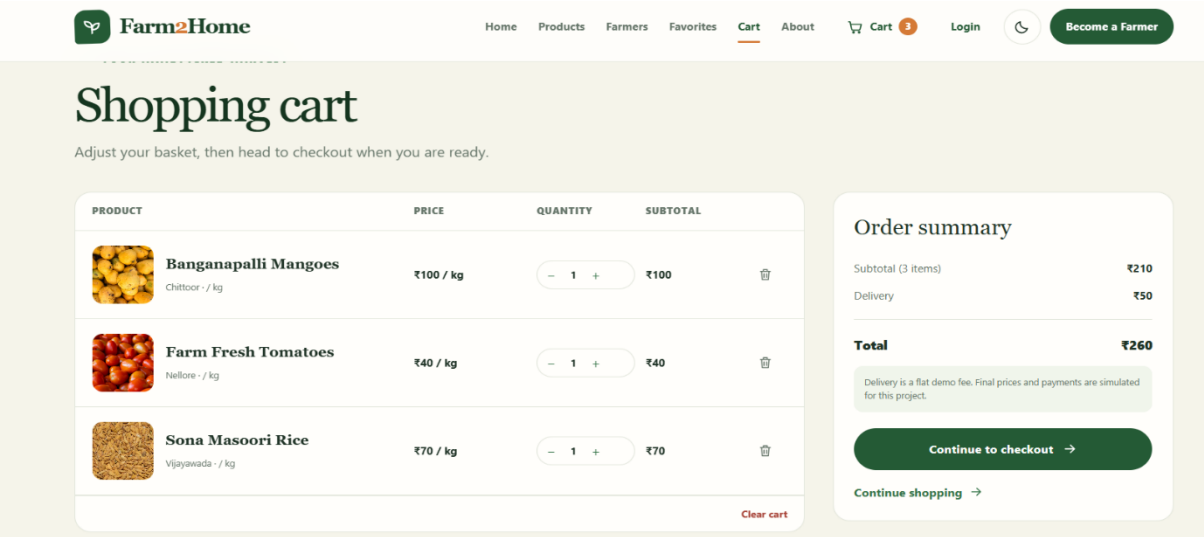
**Design principles**

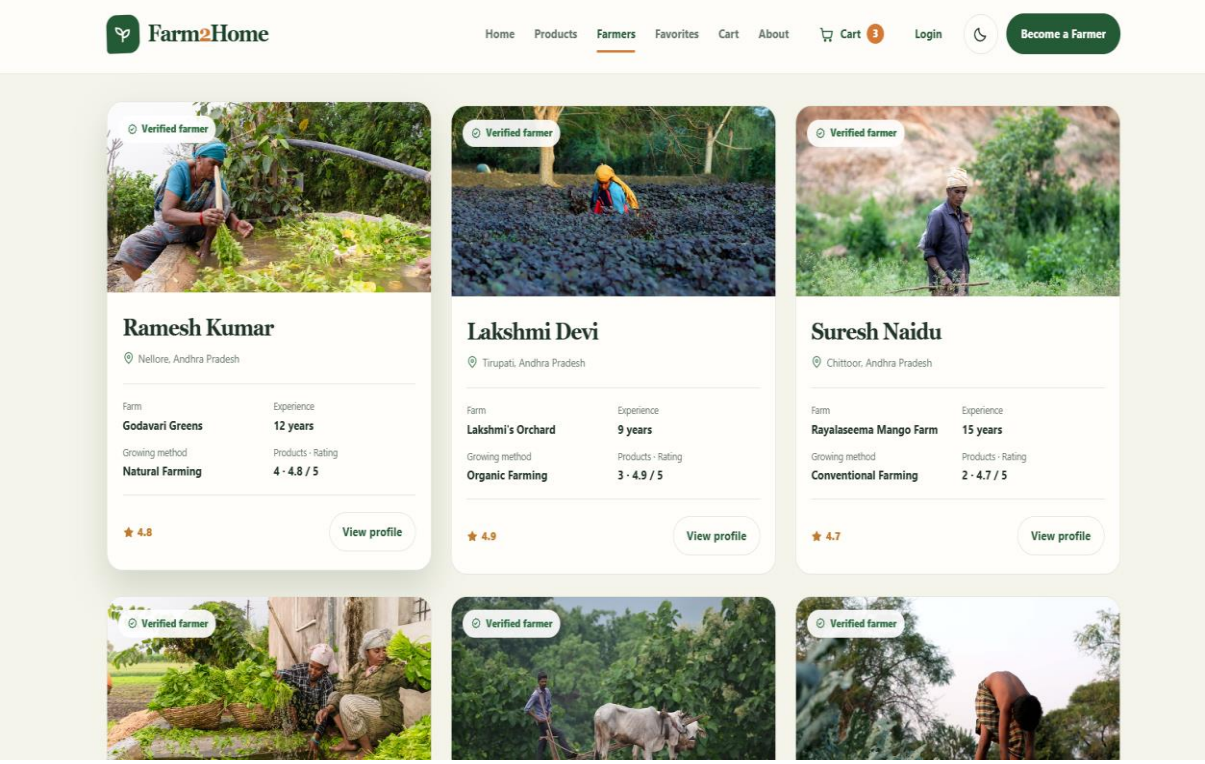
- **Consistency** - the same colours, fonts and button styles on every page.
- **Responsiveness** - CSS Grid and Flexbox with media queries make product cards flow from four columns on desktop to one column on a mobile phone.
- **Feedback** - every action (added to cart, login failed, order placed) shows a short message.
- **Simplicity** - minimum steps to buy: browse, add to cart, checkout.

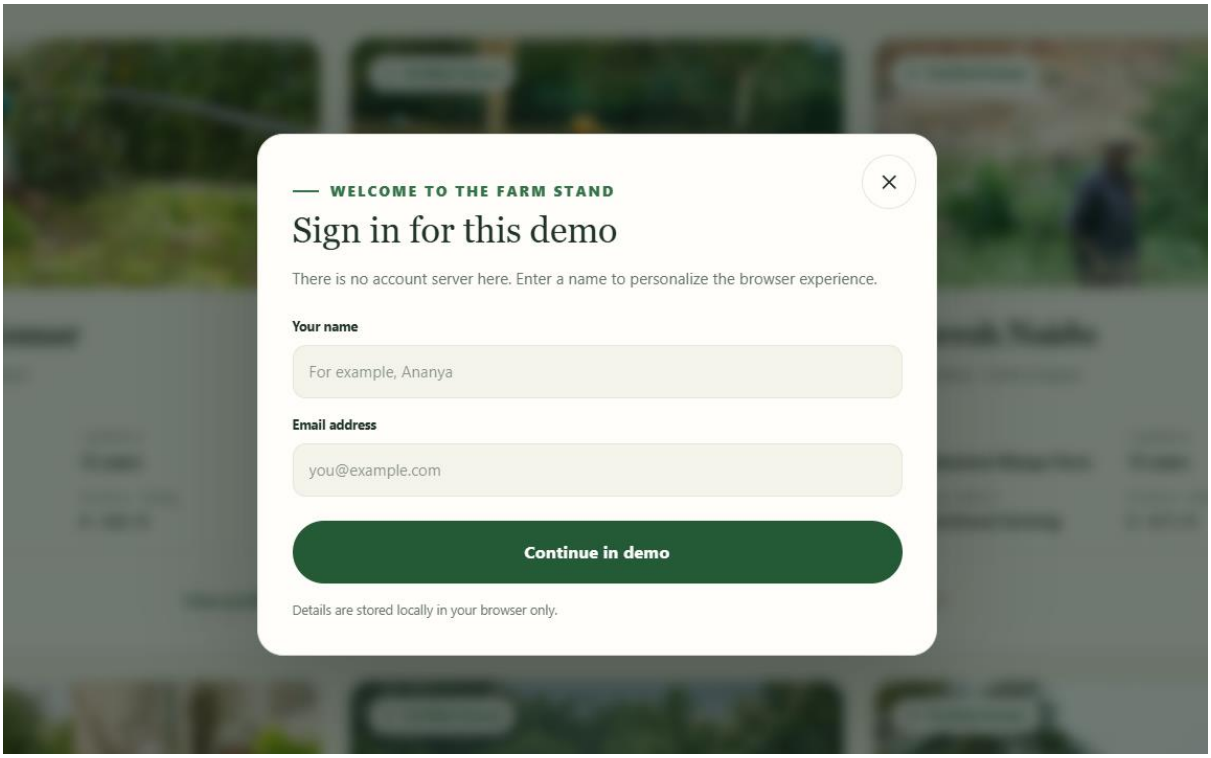
**Pages of the application**

| Page | Contents |
|---|---|
| Home / Products (Figure 6) | Banner, search box, category filter and grid of product cards with Add to Cart and Wishlist buttons |
| Login & Registration (Figure 7) | Two forms with validation messages |
| Wishlist | List of saved products with Move to Cart and Remove |
| Cart (Figure 8) | Table of items, quantity controls, subtotal, delivery charge and total |
| Checkout (Figure 8) | Delivery details form, order summary, Place Order button |
| Orders | History of orders with status |
| Profile | View and edit user details |
| Farmer Dashboard (Figure 9) | Form to add product and a table of own products with Edit, Delete and Stock toggle |


Farm2Home
Home
Products
Farmers
Favorites
Cart
About
Cart 0
Login
Become a Farmer
PICKED FOR YOUR KITCHEN
Fresh from our farmers
Harvested with care, priced transparently, and ready for your table.
See the full harvest
Fresh
FRUITS
4.9
Banganapalli Mangoes
Suresh Naidu · Chittoor
Harvested 18 Jun 2026
₹100 / kg
Add to Cart
View Details
Fresh
VEGETABLES
4.7
Farm Fresh Tomatoes
Ramesh Kumar · Nellore
Harvested 14 Jun 2026
₹40 / kg
Add to Cart
View Details
Fresh
GRAINS
4.8
Sona Masoori Rice
Fatima Begum · Vijayawada
Harvested 26 May 2026
₹70 / kg
Add to Cart
View Details
Fresh
HONEY
4.9
Wildflower Honey
Anita Reddy · Visakhapatnam
Harvested 30 May 2026
₹320 / jar
Add to Cart
View Details


Farm2Home
Home
Products
Farmers
Favorites
Cart
About
Cart 3
Login
Become a Farmer
Shopping cart
Adjust your basket, then head to checkout when you are ready.
PRODUCT
PRICE
QUANTITY
SUBTOTAL
Banganapalli Mangoes
Chittoor · / kg
₹100 / kg
1
₹100
Farm Fresh Tomatoes
Nellore · / kg
₹40 / kg
1
₹40
Sona Masoori Rice
Vijayawada · / kg
₹70 / kg
1
₹70
Clear cart
Order summary
Subtotal (3 items)
₹210
Delivery
₹50
Total
₹260
Delivery is a flat demo fee. Final prices and payments are simulated for this project.
Continue to checkout
Continue shopping


Farm2Home
Home
Products
Farmers
Favorites
Cart
About
Cart 3
Login
Become a Farmer
Verified farmer
Ramesh Kumar
Nellore, Andhra Pradesh
Farm
Godavari Greens
Experience
12 years
Growing method
Natural Farming
Products · Rating
4 · 4.8 / 5
4.8
View profile
Verified farmer
Lakshmi Devi
Tirupati, Andhra Pradesh
Farm
Lakshmi's Orchard
Experience
9 years
Growing method
Organic Farming
Products · Rating
3 · 4.9 / 5
4.9
View profile
Verified farmer
Suresh Naidu
Chittoor, Andhra Pradesh
Farm
Rayalaseema Mango Farm
Experience
15 years
Growing method
Conventional Farming
Products · Rating
2 · 4.7 / 5
4.7
View profile
Verified farmer
Verified farmer
Verified farmer


WELCOME TO THE FARM STAND
Sign in for this demo
There is no account server here. Enter a name to personalize the browser experience.
Your name
For example, Ananya
Email address
you@example.com
Continue in demo
Details are stored locally in your browser only.

# 4. CODING

This chapter presents the important code of the application. The code is divided into the JavaScript modules shown in Figure 5. The listings are representative of the logic implemented in each module.

## 4.1 Storage Helper (storage.js)

All reading and writing of browser storage is done through small helper functions so that JSON conversion and error handling are written only once.

```
// storage.js - helpers for localStorage and sessionStorage
function getData(key, fallback = []) {
  try {
    const raw = localStorage.getItem(key);
    return raw ? JSON.parse(raw) : fallback;
  } catch (e) {
    console.error('Read error', e);
    return fallback;
  }
}

function setData(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
  } catch (e) {
    alert('Storage is full or disabled in this browser.');
  }
}

function getSession() {
  const raw = sessionStorage.getItem('f2h_session');
  return raw ? JSON.parse(raw) : null;
}

function setSession(user) {
  sessionStorage.setItem('f2h_session', JSON.stringify(user));
}

function clearSession() {
  sessionStorage.removeItem('f2h_session');
}
```

## 4.2 Registration and Login (auth.js)

```
// auth.js - user registration, login and logout
function registerUser(name, email, phone, password, role) {
  const users = getData('f2h_users');
  const emailOk = email.includes('@') && email.includes('.');
  const phoneOk = phone.length === 10 && !isNaN(phone);

  if (!name || !emailOk || !phoneOk || password.length < 6) {
    return { ok: false, msg: 'Please enter valid details.' };
  }
  if (users.some(u => u.email === email)) {
    return { ok: false, msg: 'This e-mail is already registered.' };
  }

  const user = { id: 'U' + Date.now(), name, email, phone,
                 password, role: role || 'consumer', address: '' };
```

```
  users.push(user);
  setData('f2h_users', users);
  return { ok: true, msg: 'Registration successful. Please log in.' };
}

function loginUser(email, password) {
  const user = getData('f2h_users')
    .find(u => u.email === email && u.password === password);
  if (!user) return { ok: false, msg: 'Invalid e-mail or password.' };
  setSession(user);
  return { ok: true, role: user.role };
}

function logoutUser() {
  clearSession();
  window.location.href = 'index.html';
}
```

## 4.3 Loading, Rendering and Searching Products (products.js)

On the first load the catalogue is created from a default array of sample products. After that the list is always read from localStorage, so products added by farmers appear automatically.

```
// products.js - catalogue, rendering, search and filter
const defaultProducts = [
  { id: 'P1', name: 'Tomato', category: 'Vegetables', price: 30, unit: 'kg',
    stock: 100, available: true, farmer: 'Ramu', image: 'images/tomato.jpg',
    description: 'Fresh red tomatoes from local farms.' },
  { id: 'P2', name: 'Banana', category: 'Fruits', price: 50, unit: 'dozen',
    stock: 60, available: true, farmer: 'Lakshmi', image: 'images/banana.jpg',
    description: 'Naturally ripened bananas.' }
  // ... more products
];

function loadProducts() {
  let list = getData('f2h_products', null);
  if (!list) {
    list = defaultProducts;
    setData('f2h_products', list);
  }
  return list;
}

function renderProducts(list) {
  const grid = document.getElementById('productGrid');
  grid.innerHTML = '';
  if (list.length === 0) {
    grid.innerHTML = '<p class="empty">No products found.</p>';
    return;
  }
  list.forEach(p => {
    const card = document.createElement('div');
    card.className = 'card';
    card.innerHTML = `
      <img src="${p.image}" alt="${p.name}">
      <h3>${p.name}</h3>
      <p class="cat">${p.category} | ${p.farmer}</p>
      <p class="price">Rs.${p.price} / ${p.unit}</p>
      <p class="${p.available ? 'in' : 'out'}">
        ${p.available ? 'In Stock' : 'Out of Stock'}</p>
      <button onclick="toggleWishlist('${p.id}')">Wishlist</button>
      <button ${p.available ? '' : 'disabled'}
              onclick="addToCart('${p.id}')">Add to Cart</button>`;
    grid.appendChild(card);
  });
}
```

```
function applyFilters() {
  const text = document.getElementById('searchBox').value.toLowerCase();
  const cat  = document.getElementById('categorySelect').value;
  const result = loadProducts().filter(p =>
    p.name.toLowerCase().includes(text) &&
    (cat === 'All' || p.category === cat));
  renderProducts(result);
}

document.getElementById('searchBox').addEventListener('input', applyFilters);
document.getElementById('categorySelect').addEventListener('change', applyFilters);
window.addEventListener('DOMContentLoaded', () => renderProducts(loadProducts()));
```

## 4.4 Wishlist (wishlist.js)

```
// wishlist.js - per-user wishlist
function toggleWishlist(productId) {
  const user = getSession();
  if (!user) { alert('Please log in first.'); return; }

  const key = 'f2h_wishlist_' + user.id;
  let list = getData(key);
  if (list.includes(productId)) {
    list = list.filter(id => id !== productId);
  } else {
    list.push(productId);
  }
  setData(key, list);
  updateBadges();
}
```

## 4.5 Shopping Cart (cart.js)

```
// cart.js - cart operations and totals
function addToCart(productId) {
  const user = getSession();
  if (!user) { alert('Please log in to add items.'); return; }

  const key = 'f2h_cart_' + user.id;
  const cart = getData(key);
  const item = cart.find(i => i.productId === productId);
  if (item) item.qty += 1;
  else cart.push({ productId, qty: 1 });

  setData(key, cart);
  updateBadges();
}

function updateQuantity(productId, change) {
  const key = 'f2h_cart_' + getSession().id;
  let cart = getData(key);
  const item = cart.find(i => i.productId === productId);
  if (!item) return;
  item.qty += change;
  if (item.qty <= 0) cart = cart.filter(i => i.productId !== productId);
  setData(key, cart);
  renderCart();
}

function calculateTotal() {
  const products = loadProducts();
  const cart = getData('f2h_cart_' + getSession().id);
  let subtotal = 0;
  cart.forEach(i => {
```

```
    const p = products.find(x => x.id === i.productId);
    if (p) subtotal += p.price * i.qty;
  });
  const delivery = subtotal > 0 && subtotal < 300 ? 30 : 0;
  return { subtotal, delivery, total: subtotal + delivery };
}

function updateBadges() {
  const user = getSession();
  if (!user) return;
  const count = getData('f2h_cart_' + user.id)
                  .reduce((n, i) => n + i.qty, 0);
  document.getElementById('cartCount').textContent = count;
}
```

## 4.6 Checkout and Order Placement (checkout.js)

```
// checkout.js - validation and order creation
function validateCheckout(name, phone, address) {
  if (!name.trim()) return 'Enter the receiver name.';
  if (phone.length !== 10 || isNaN(phone)) return 'Enter a valid 10-digit phone
number.';
  if (address.trim().length < 10) return 'Enter a complete address.';
  return '';
}

function generateOrderId() {
  const orders = getData('f2h_orders');
  return 'F2H-' + (1000 + orders.length + 1);
}

function placeOrder(name, phone, address) {
  const user = getSession();
  const error = validateCheckout(name, phone, address);
  if (error) { alert(error); return; }

  const key = 'f2h_cart_' + user.id;
  const cart = getData(key);
  if (cart.length === 0) { alert('Your cart is empty.'); return; }

  const products = loadProducts();
  const items = cart.map(i => {
    const p = products.find(x => x.id === i.productId);
    return { productId: p.id, name: p.name, price: p.price, qty: i.qty };
  });

  const order = {
    orderId: generateOrderId(), userId: user.id, items,
    total: calculateTotal().total, address,
    payment: 'Cash on Delivery',
    date: new Date().toLocaleString(), status: 'Placed'
  };

  const orders = getData('f2h_orders');
  orders.push(order);
  setData('f2h_orders', orders);
  setData(key, []);                        // empty the cart
  sessionStorage.setItem('f2h_checkout', JSON.stringify(order));
  window.location.href = 'orders.html';   // show confirmation
}
```

## 4.7 Farmer Product Management (farmer.js)

```
// farmer.js - add, delete and availability
function addProduct(data) {
  const products = loadProducts();
  const farmer = getSession();
  products.push({ id: 'P' + Date.now(), ...data,
                  farmer: farmer.name, available: data.stock > 0 });
  setData('f2h_products', products);
  renderFarmerTable();
}

function deleteProduct(id) {
  if (!confirm('Delete this product?')) return;
  setData('f2h_products', loadProducts().filter(p => p.id !== id));
  renderFarmerTable();
}

function setAvailability(id, state) {
  const products = loadProducts();
  const p = products.find(x => x.id === id);
  if (p) { p.available = state; setData('f2h_products', products); }
  renderFarmerTable();
}
```

## 4.8 Responsive Styling (style.css)

```
/* style.css - responsive product grid */
:root { --green: #2e7d32; --light: #e8f5e9; --dark: #1b5e20; }

body { font-family: 'Poppins', Arial, sans-serif; margin: 0; background: #fafafa; }
header { background: var(--green); color: #fff; display: flex;
         align-items: center; justify-content: space-between; padding: 10px 24px; }

#productGrid { display: grid; gap: 20px; padding: 24px;
               grid-template-columns: repeat(4, 1fr); }
.card { background: #fff; border-radius: 10px; padding: 12px;
        box-shadow: 0 2px 8px rgba(0,0,0,.12); }
.card img { width: 100%; height: 160px; object-fit: cover; border-radius: 8px; }
.price { color: var(--dark); font-weight: 600; }
.out   { color: #c62828; }

@media (max-width: 1024px) { #productGrid { grid-template-columns: repeat(3, 1fr); } }
@media (max-width: 768px)  { #productGrid { grid-template-columns: repeat(2, 1fr); } }
@media (max-width: 480px)  { #productGrid { grid-template-columns: 1fr; } }
```

### Concepts demonstrated by the code

| Concept | Where used |
|---|---|
| DOM manipulation | renderProducts(), renderCart(), renderFarmerTable() |
| Event handling | search input event, category change event, button onclick, form submit |
| Data persistence | localStorage for catalogue, users, orders, cart, wishlist |
| Session handling | sessionStorage for the logged-in user |
| Responsive design | CSS Grid, Flexbox, media queries |
| Form validation | Length and format checks in registerUser() and validateCheckout() |

# 5. TESTING

Testing is the process of executing a program with the intention of finding errors and confirming that the software meets its requirements. For Farm2Home, testing was carried out module by module and then for the complete application in several browsers.

## 5.1 Test Plan

**Unit Testing:** Unit testing is a software development process in which the smallest testable parts of an application, called units, are individually and independently scrutinised for proper operation. In this project the units are the JavaScript functions such as registerUser(), addToCart(), calculateTotal() and validateCheckout(). Each unit was tested manually with valid and invalid inputs and the output was compared with the expected result.

**Integration Testing:** After the individual functions were verified, modules were combined and tested together, for example: login followed by add to cart, then cart total, then checkout, then order history. This confirmed that data written by one module (for example the cart) is correctly read by the next module (for example checkout).

**System Testing:** The complete application was tested from the point of view of a user for the full purchase scenario and for the farmer scenario, using the functional requirements of Chapter 2 as the checklist.

**Compatibility and Responsive Testing:** The application was opened in Google Chrome, Mozilla Firefox and Microsoft Edge on desktop and in Chrome on an Android phone. The browser developer tools were used to simulate screen widths of 320 px, 480 px, 768 px, 1024 px and 1920 px.

**Test environment**

| Item | Details |
|---|---|
| Operating System | Windows 10 / 11 |
| Browsers | Chrome, Firefox, Edge, Chrome for Android |
| Editor | Visual Studio Code with Live Server |
| Testing method | Manual black-box testing with browser developer console |

## 5.2 Test Report

The following test cases were executed. Each test case lists the module, the input, the expected result and the outcome observed.

| TC No | Module | Test Case / Input | Expected Result | Status |
|---|---|---|---|---|
| TC01 | Registration | All fields valid | Account created, success message shown | Pass |
| TC02 | Registration | Empty name field | Error message: enter valid details | Pass |
| TC03 | Registration | E-mail without @ symbol | Error message shown, no account created | Pass |
| TC04 | Registration | Phone number with 8 digits | Error message shown | Pass |
| TC05 | Registration | Password shorter than 6 characters | Error message shown | Pass |
| TC06 | Registration | E-mail that is already registered | Error: e-mail already registered | Pass |
| TC07 | Login | Correct e-mail and password | User logged in, name shown in header | Pass |
| TC08 | Login | Wrong password | Error: invalid e-mail or password | Pass |
| TC09 | Logout | Click Logout | Session cleared, home page opens | Pass |
| TC10 | Products | Open home page first time | Default products loaded and displayed | Pass |
| TC11 | Search | Type tom in search box | Only Tomato card is displayed | Pass |
| TC12 | Search | Type a word with no match | No products found message | Pass |
| TC13 | Filter | Select category Fruits | Only fruit products displayed | Pass |
| TC14 | Wishlist | Click Wishlist while logged out | Alert asking user to log in | Pass |
| TC15 | Wishlist | Click Wishlist on a product (logged in) | Product added to wishlist | Pass |
| TC16 | Wishlist | Click Wishlist on the same product again | Product removed from wishlist | Pass |
| TC17 | Cart | Add product while logged out | Alert asking user to log in | Pass |
| TC18 | Cart | Add product (logged in) | Cart count increases by one | Pass |
| TC19 | Cart | Add the same product twice | Quantity becomes two, one row only | Pass |
| TC20 | Cart | Increase quantity with + button | Quantity and total updated | Pass |
| TC21 | Cart | Decrease quantity to zero | Item removed from cart | Pass |
| TC22 | Cart total | Subtotal below 300 | Delivery charge of 30 added | Pass |
| TC23 | Cart total | Subtotal 300 or above | Delivery charge is zero | Pass |

| TC No | Module | Test Case / Input | Expected Result | Status |
|---|---|---|---|---|
| TC24 | Out of stock | Product marked Out of Stock | Add to Cart button is disabled | Pass |
| TC25 | Checkout | Empty receiver name | Error message shown, order not placed | Pass |
| TC26 | Checkout | Invalid phone number | Error message shown | Pass |
| TC27 | Checkout | Empty cart | Alert: your cart is empty | Pass |
| TC28 | Checkout | All details valid | Order created, cart emptied, confirmation shown | Pass |
| TC29 | Orders | Open My Orders after an order | New order listed with id, date, total, status | Pass |
| TC30 | Persistence | Close and reopen the browser | Users, products, orders still available | Pass |
| TC31 | Session | Close the tab and reopen | User must log in again | Pass |
| TC32 | Farmer | Add a new product | Product appears in farmer table and on home page | Pass |
| TC33 | Farmer | Delete a product | Product removed from both pages | Pass |
| TC34 | Farmer | Toggle availability to Out of Stock | Consumers see Out of Stock | Pass |
| TC35 | Responsive | Screen width 480 px | Single-column layout, no horizontal scroll | Pass |

*Figure 10. Test Report - [insert screenshot of the test run / browser console here]*

**Summary of results**

| Total test cases | Passed | Failed |
|---|---|---|
| 35 | 35 | 0 |

**Defects found and fixed during testing**

- Cart count badge did not update after removing an item; fixed by calling updateBadges() inside updateQuantity().
- The same e-mail could be registered in upper and lower case; fixed by converting e-mail to lower case before comparison.
- Product cards overflowed on narrow screens; fixed by adding the 480 px media query.

# 6. INSTALLATION INSTRUCTIONS

Farm2Home is a static web application. It does not need any server, database or package installation. The prerequisites are only a modern web browser and, optionally, a code editor.

**Prerequisites**

- A modern web browser (Chrome, Firefox, Edge or Safari) with JavaScript enabled.
- Visual Studio Code with the Live Server extension (optional, for development).
- Git (optional, to clone the repository).

**Steps to run on a local machine**

1. Copy the project folder Farm2Home to your computer (or clone it using git: git clone <repository-url>).
2. Make sure the folder structure is the same as given in Figure 5 (index.html, css, js and images folders).
3. Open the folder in Visual Studio Code.
4. Right-click index.html and choose Open with Live Server. Alternatively, double-click index.html to open it directly in the browser.
5. The Farm2Home home page opens and the default products are loaded automatically.
6. Register a new account, or log in with the sample accounts if they are provided in the default data.

**Steps to host online (optional)**

1. Create a repository on GitHub and upload all the project files.
2. Open Settings, then Pages, and choose the main branch as the source.
3. After a short time the site is available at the GitHub Pages address.

**Resetting the application data**

Open the browser developer tools (F12), go to Application, then Local Storage, and delete the keys beginning with f2h_ . Alternatively run localStorage.clear() in the console and reload the page.

# 7. END-USER INSTRUCTIONS

This chapter explains how a consumer and a farmer should use Farm2Home.

## 7.1 For the Consumer

1. **Open the website.** The home page shows the available products in the form of cards.
2. **Register.** Click Login / Register, choose Register, fill in the name, e-mail, phone number and a password of at least six characters, and press Register.
3. **Log in.** Enter the e-mail and the password and press Login. The header then shows your name.
4. **Search and filter.** Type the product name in the search box or choose a category such as Vegetables or Fruits. The list changes instantly.
5. **View details.** Each card shows the price, unit, farmer name and availability.
6. **Add to Wishlist.** Click the Wishlist button to save a product for later. Open the Wishlist page to move it to the cart or remove it.
7. **Add to Cart.** Click Add to Cart. The cart icon shows the number of items. Out-of-stock products cannot be added.
8. **Manage the cart.** Open the Cart page, use the + and - buttons to change the quantity or remove an item. The total is shown at the bottom.
9. **Checkout.** Press Proceed to Checkout, enter the receiver name, phone number and delivery address and press Place Order.
10. **Confirmation.** An order id is displayed. The order can be seen later in My Orders.
11. **Logout.** Always press Logout when using a shared computer.

## 7.2 For the Farmer

1. Register with the role Farmer and log in.
2. The Farmer Dashboard opens with a form and a table.
3. Fill in the product name, category, price, unit, quantity and description and press Add Product.
4. The product appears in the table and on the consumer home page.
5. Use Edit to change price or quantity, Delete to remove a product, and the stock toggle to mark a product In Stock or Out of Stock.

**Important points for users**

- Data is saved in your own browser. Do not clear the browser data if you want to keep your account and orders.
- Use the same browser and device to see the same data.
- This is a prototype: do not enter real card or bank details. The only payment mode is Cash on Delivery.

# 8. FUTURE WORK

Some of the possible amendments and improvements in this system are:

- Adding a server-side backend and database
- Online payment gateway integration
- Real-time order tracking and delivery management
- Ratings, reviews and farmer profiles
- Multi-language support and voice search
- Machine learning based price and demand prediction

**Server-side backend and database.** The most important improvement is to replace the browser storage with a real backend, for example Node.js with Express and MongoDB, or Python Flask with MySQL. Then the data of all farmers and consumers will be shared across devices, and passwords can be stored securely using hashing and authentication tokens (JWT).

**Online payment.** A payment gateway such as Razorpay or UPI can be integrated so that consumers can pay online securely, in addition to Cash on Delivery.

**Order tracking and delivery.** Order status can be updated by the farmer or delivery partner and shown to the consumer as Placed, Packed, Out for Delivery and Delivered, with SMS or e-mail notifications.

**Ratings and reviews.** Consumers can rate products and farmers. This will build trust and help other customers choose good quality produce.

**Language support and accessibility.** Many farmers are comfortable in Telugu, Hindi or other regional languages. Adding multiple languages and voice input will make the application usable by a larger group.

**Machine learning.** Since the department is Computer Science and Engineering (AI/ML), the platform can be extended with ML models for demand forecasting, fair-price recommendation to the farmer, and personalised product recommendations to the consumer based on order history.

# 9. SUMMARY

Farm2Home demonstrates how a complete farm-to-consumer shopping experience can be built using only HTML, CSS and JavaScript. The application allows consumers to register, browse, search, filter and view fresh farm products, maintain a wishlist and cart, and place orders through a checkout module. A farmer-oriented interface allows farmers to display and manage their products so that produce reaches the customer directly, reducing the gap between farmers and consumers.

By using the browser's localStorage and sessionStorage, the application maintains user, product, cart, wishlist and order information without a server-side backend, which keeps the system simple to run, test and demonstrate. The project covers important front-end concepts: DOM manipulation, event handling, responsive design, form validation, data persistence and dynamic interface generation. Testing of the modules and of the complete flow showed that the application meets the functional requirements defined in the requirement specification.

Although the present version is a prototype with limitations such as local-only data and simulated payment, it provides a strong foundation. With a backend, secure authentication, online payments and delivery tracking, Farm2Home can grow into a real direct farm-to-consumer marketplace that benefits farmers with better prices and consumers with fresher and more transparent produce.

# 10. REFERENCES


- MDN Web Docs - HTML: https://developer.mozilla.org/en-US/docs/Web/HTML
- MDN Web Docs - CSS: https://developer.mozilla.org/en-US/docs/Web/CSS
- MDN Web Docs - JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDN Web Docs - Web Storage API (localStorage and sessionStorage): https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- MDN Web Docs - Document Object Model (DOM): https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- W3Schools HTML, CSS and JavaScript tutorials: https://www.w3schools.com/
- W3C Web Content Accessibility Guidelines (WCAG): https://www.w3.org/WAI/standards-guidelines/wcag/
- Visual Studio Code Documentation: https://code.visualstudio.com/docs
- GitHub Pages Documentation: https://docs.github.com/en/pages
- Pressman, R. S., Software Engineering: A Practitioner's Approach, McGraw-Hill.
- Sommerville, I., Software Engineering, Pearson.
- Flanagan, D., JavaScript: The Definitive Guide, O'Reilly Media.